QATIP Intermediate
Azure Lab03a
Service Principals

Contents

Overview 1
Before You Begin 1
Phase 0 – Create a Limited-Service Principal 2
Phase 1 – Run Terraform With Limited SP Identity 3
Phase 2 – Assign Contributor Role to the Service Principal 4

Overview

This lab introduces secure, least-privilege access using **Azure Service Principals (SPs)**. You will:

- Create a Service Principal with limited permissions scoped to the subscription
- Use SP credentials via environment variables
- Run Terraform operations and observe RBAC restrictions
- Upgrade permissions and reattempt deployments

Before You Begin

1. Navigate to the lab directory:

```
cd c:\azure-tf-int\labs\03a
```


©QA 2026

Phase 0 – Create a Limited-Service Principal

Purpose

What we are doing:
We are creating a new **non-human identity** (Service Principal) that will be used to run Terraform, representing a **limited automation identity** — as might be used in CI/CD pipelines.

Why it matters:
Rather than using our full-access lab account, we want to simulate **realistic access boundaries** — mimicking how secure organizations prevent developers and automation tools from having unrestricted rights.

Environment variable usage:
Instead of hardcoding secrets, we inject them into the environment. This aligns with DevSecOps best practices and is supported natively by the Terraform AzureRM provider.

Steps

1. Update line 2 of **main.tf**, replacing **<your subscription id>** with your lab subscription id.

2. Create the Service Principal with Reader access, scoped to your subscription (replace **<sub_id>** with your lab subscription and **<4 digits>** with 4 random digits of your choice):

   az ad sp create-for-rbac --name terraform-sp-<4 digits> --role Reader --scopes /subscriptions/<sub_id>

3. Note down the displayed output as you will need it later and then create the following environment variables;

   $env:ARM_CLIENT_ID = "<appId>"
   $env:ARM_CLIENT_SECRET = "<password>"
   $env:ARM_SUBSCRIPTION_ID = "<your subscription id>"
   $env:ARM_TENANT_ID = "<tenant>"

4. Adopt the SP identity:

   az logout

   az login --service-principal --username $env:ARM_CLIENT_ID --password=$env:ARM_CLIENT_SECRET --tenant $env:ARM_TENANT_ID

   You should see "type": "servicePrincipal" in the output.

Phase 1 – Run Terraform With Limited SP Identity

Purpose

What we are doing:
Attempting to deploy infrastructure using Terraform under this limited SP identity.

Why it matters:
This phase is expected to fail — and that is intentional. It highlights that simply authenticating to Azure is not enough; **role assignments define what you are allowed to do**. It also encourages developers to review error messages meaningfully and connect failures to permissions.

Steps

1. Run

   terraform init
   terraform plan
   terraform apply --auto-approve

   Note: This lab uses Azure provider version 4.16 which will automatically register the networking resource provider. If using version >5.0, you will receive error messages regarding registration of the Azure networking resource provider, run the following command before retrying the plan and apply;

   az provider register --namespace Microsoft.Network

2. Expect Terraform apply to **fail** with authorization errors.

Phase 2 – Assign Contributor Role to the Service Principal

Purpose

What we are doing:
Temporarily switching back to the lab account to assign Contributor rights to the SP.

Why it matters:
This demonstrates the **RBAC model in action** — where a principal's capabilities change dynamically based on role assignments. It mirrors how platform teams might approve temporary elevated access to unblock automation or development pipelines.

Steps

1. Re-adopt your lab identity:

   az logout
   az login

2. Inserting your App id and Subscription id, run:

   az role assignment create --assignee <appId> --role Contributor --scope /subscriptions/<sub_id>

3. Re-adopt the SP identity:

   az logout

   az login --service-principal --username $env:ARM_CLIENT_ID --password=$env:ARM_CLIENT_SECRET --tenant $env:ARM_TENANT_ID

4. Retry:

terraform plan
terraform apply --auto-approve

5. Terraform should now **succeed**. Use the Azure Portal to view the new resources.

   Notes: If you receive errors regarding subscription registration in Denmark East, use the following command to list available regions and update main.tf accordingly before retrying the plan and apply;

   az account list-locations --query "[].{Region:name,DisplayName:displayName}" --output table

6. Remove the deployed resources; **terraform destroy --auto-approve**

7. Identify the Service Principal object id (replace <appid> with your App id)...

   az ad sp show --id <appid> --query "{ objectId:id}" -o table

8. Attempt to delete the Service Principal and App registration...

   az ad sp delete --id <object id>
   az ad app delete --id <appid>

   The deletion attempts should fail because the Service Principal being used does not have sufficient permissions.

9. Re-adopt your lab identity:

   az logout
   az login

10. Delete the Service Principal... **az ad sp delete --id <object id>**

11. Delete the App registration... **az ad app delete --id <appid>**

12. Delete local environment variables... **Remove-Item Env:ARM_***

Congratulations, you have completed this lab

Copyright

© 2026 QA Michael Coulling-Green. All rights reserved. These lab materials are provided as part of an authorised training course. Course attendees may reuse these materials for their own personal learning and reference outside of the training session. Unauthorised copying, redistribution, publication, or use of these materials to deliver training is prohibited without prior written permission from the author.

© QA 2026